

Emmet in VS Code

A practical HTML shortcut reference

June 16, 2026

Contents

1	Getting Started	2
1.1	The HTML document skeleton	3
2	Classes and IDs	4
3	Operators and Abbreviation Structure	4
3.1	Combining operators	5
4	Lists, Text, and Numbering	5
4.1	What repetition does and does not do	6
5	Form Inputs	7
5.1	Building a more complete form	7
6	Semantic HTML	8
7	Practical Patterns	9
7.1	A quick editing checklist	9
8	Conclusion	10
	References	11

1 Getting Started

Emmet is a shorthand language for writing HTML and other structured markup. In Visual Studio Code, type an abbreviation in an HTML file and expand it to produce the corresponding elements. This is especially useful for repeated structures: instead of typing every opening tag, closing tag, and attribute by hand, describe the shape once and let the editor build it.

In a standard VS Code setup, the usual expansion key is **Tab**. Depending on your editor settings and suggestion list, **Enter** may also accept an Emmet suggestion. If an abbreviation does not expand, first check that the file is recognized as HTML, then try Tab or select the Emmet suggestion explicitly. The examples below use Enter where the supplied shortcut list specifies it; the expansion itself is the important part.

Emmet generates a starting structure, not a finished page. Review the result, fill in meaningful text and attribute values, and check that the markup suits the page's purpose. For example, an image needs useful alternative text, and a form needs labels that identify its controls.

In VS Code	Press	Result
!	Enter or Tab	HTML document skeleton with doctype, head, metadata, title, and body
html	Enter or Tab	<html></html>
head	Enter or Tab	<head></head>
body	Enter or Tab	<body></body>
div	Enter or Tab	
p	Enter or Tab	
h1	Enter or Tab	
a	Enter or Tab	<ahref="">
img	Enter or Tab	<imgsrc=""alt="">
br, hr	Enter or Tab	 or <hr>
button	Enter or Tab	<button></button>
form	Enter or Tab	<formaction=""></form>
label	Enter or Tab	<labelfor=""></label>
input	Enter or Tab	<inputtype="text">
textarea	Enter or Tab	<textarea></textarea>
select	Enter or Tab	<selectname=""id=""></select>
option	Enter or Tab	<optionvalue=""></option>

1.1 The HTML document skeleton

The ! abbreviation is a convenient starting point for a new HTML file. It creates a document declaration and a basic page structure, including character encoding and a viewport setting that helps the page display appropriately on mobile screens. The generated title is a placeholder; replace it with a useful title that describes the page. Likewise, set the document language to match the page's primary language rather than leaving a default value that may not fit the content.

The remaining element shortcuts are useful when building the page inside that skeleton. A heading, paragraph, link, or form control can be inserted with a short abbreviation and then edited in place. Emmet reduces repetitive typing, but it does not decide whether a particular element is semantically appropriate. Use headings to express doc-

ument structure, links for navigation, and buttons for actions.

2 Classes and IDs

Classes and IDs are added directly to an abbreviation using a dot or hash. A dot introduces a class name; a hash introduces an ID. When the abbreviation starts with a class or ID alone, Emmet assumes a `div`. When a tag is written first, the class or ID is attached to that element instead. This makes common structures quick to express without typing the full opening and closing tags.

Use classes for styling or identifying groups of elements that share a role. An ID should identify a unique element in the document. These conventions are useful well beyond the abbreviation itself: readable names make the resulting HTML easier to maintain and help keep CSS selectors understandable.

Shortcut	Generated HTML
<code>.box</code>	<code><divclass="box"></div></code>
<code>\#header</code>	<code><divid="header"></div></code>
<code>div.box</code>	<code><divclass="box"></div></code>
<code>p.text</code>	<code><pclass="text"></p></code>
<code>div\#main</code>	<code><divid="main"></div></code>
<code>div.container.box</code>	<code><divclass="containerbox"></div></code>

3 Operators and Abbreviation Structure

Emmet operators express relationships between elements. The child operator nests an element inside another; the sibling operator places elements next to each other. Repetition creates several copies, while grouping makes it possible to apply an operator to a compact substructure before continuing with another element. These operators are most powerful when combined: a short expression can describe a complete set of nested elements.

Read an abbreviation from left to right, paying attention to the operator between each part. In `div>p`, the paragraph is a child of the division. In `h1+p`, the heading and paragraph are siblings. If an expression becomes difficult to read, add grouping parentheses or expand it in smaller steps. A few extra keystrokes are preferable to an abbreviation whose structure is hard to reason about.

Symbol	Meaning	Example	Result
>	Child element	div>p	Paragraph nested inside a div
+	Sibling elements	h1+p	Heading followed by a paragraph
*	Repeat elements	li*5	Five li elements
.	Add a class	.card	<divclass="card"></div>
\#	Add an ID	\#main	<divid="main"></div>
\{\}	Insert text	p\{Hello\}	Hello
[]	Add an attribute	input[type=text]	<inputtype="text">
\\$	Number repeated items	li.item\\$*3	Class names numbered from item1 to item3
()	Group expressions	(header>nav)+main	Header with navigation, followed by main

3.1 Combining operators

A useful way to learn the operators is to begin with one relationship and add another. For example, `div>p` establishes nesting. Adding a sibling with `+` gives `div>h2+p`; adding repetition with `*` makes `ul>li*5`. This incremental approach makes it easier to spot a misplaced operator before expanding a large expression.

Grouping is particularly helpful when one structure must be repeated or followed by a sibling. Parentheses mark the part of the abbreviation that belongs together. For instance, `(header>nav)+main` describes two top-level elements: a header containing navigation, followed by a main region. Once expanded, inspect the indentation and nesting. The generated HTML should reflect the intended document hierarchy, not simply the shortest expression you could type.

4 Lists, Text, and Numbering

List abbreviations combine nesting and repetition. The expression `ul>li*5` creates an unordered list with five empty list items; `ol>li*5` creates the same structure as an

ordered list. To make a navigation list, add a class to the list and nest a link inside each repeated item. Emmet repeats the complete child structure for every item.

Text in braces is inserted as literal content. Numbering with a dollar sign is useful for producing distinct labels or class names across repeated elements. A numbered abbreviation can save time when creating a scaffold, but the generated text is still generic. Replace placeholder labels with content that makes sense to readers, and ensure repeated links have meaningful destinations.

Shortcut	Generated structure	Typical use
<code>ul>li</code>	One unordered list with one list item	A short bulleted list
<code>ul>li*5</code>	One unordered list with five list items	A list scaffold
<code>ol>li*5</code>	One ordered list with five list items	Steps or ranked items
<code>ul.menu>li*4>a</code>	A list with class menu; four items, each containing a link	Navigation links
<code>p\{Hello\}</code>	<code><p>Hello</p></code>	A paragraph with starter text
<code>h1\{Welcome\}</code>	<code><h1>Welcome</h1></code>	A heading with starter text
<code>p\{Item\\$\}*5</code>	Five paragraphs labeled Item 1 through Item 5	Repeated placeholder copy
<code>li.item\\$*5</code>	Five list items with classes item1 to item5	Distinct repeated class names
<code>h1.title\{Heading\}</code>	<code><h1class="title">Heading</h1></code>	A named heading element

4.1 What repetition does and does not do

Repetition creates markup, not meaningful content. For example, `p\{Item\$\}*5` produces five paragraphs with numbered placeholder text, but it does not create five useful descriptions. Similarly, `ul.menu>li*4>a` gives each link an empty href value and no visible label. Those empty values are a prompt to complete the navigation, not a finished accessible menu.

After expansion, add the required text and attributes. For a link, provide a destination and a concise label that communicates where it leads. For a list, choose ordered or unordered markup according to whether the sequence matters. This short review

is worth doing: generated code is fast to produce, but good structure still depends on deliberate editing.

5 Form Inputs

Emmet offers shortcuts for common HTML input types. These abbreviations create the input element with its type set, making it quicker to scaffold forms for text, email, passwords, numbers, dates, and other common data. The generated element is only one part of a usable form. Add an associated label, provide any required name or validation attributes, and decide whether the control is required or optional.

A type should match the data the user is expected to enter. For instance, an email input can provide a suitable mobile keyboard and basic browser validation, while a date input requests a date-oriented control. Do not rely on placeholder text as the only label: visible, programmatically associated labels help users understand each field and support assistive technology.

Shortcut	Generated input	Common purpose
<code>input:text</code>	<code><input type="text"></code>	Short text entry
<code>input:email</code>	<code><input type="email"></code>	Email address
<code>input:password</code>	<code><input type="password"></code>	Password entry
<code>input:number</code>	<code><input type="number"></code>	Numeric value
<code>input:checkbox</code>	<code><input type="checkbox"></code>	Independent choice
<code>input:radio</code>	<code><input type="radio"></code>	One choice in a group
<code>input:file</code>	<code><input type="file"></code>	File selection
<code>input:submit</code>	<code><input type="submit"></code>	Form submission control
<code>input:date</code>	<code><input type="date"></code>	Date selection
<code>input:tel</code>	<code><input type="tel"></code>	Telephone number
<code>input:search</code>	<code><input type="search"></code>	Search query
<code>input:url</code>	<code><input type="url"></code>	Web address
<code>input:color</code>	<code><input type="color"></code>	Color selection

5.1 Building a more complete form

A form abbreviation can combine labels, inputs, and a button into one scaffold. For example, `form>label+input:email+label+input:password+button` generates

a form containing those elements in sequence. It does not automatically connect each label to its control, name the controls, or provide helpful button text. Those details are essential for a form that people can use.

Give each label a `for` value that matches the corresponding input's `id`. Add a `name` attribute when the form data must be submitted under a meaningful key. The button should say what action it performs. These small improvements make the generated structure more understandable to users and maintainers alike.

6 Semantic HTML

Semantic elements describe the role of the content they contain. Emmet can expand common semantic tags directly, so there is little reason to use generic containers everywhere. A `header` may introduce a page or section; `nav` groups navigation; `main` identifies the primary content; and `footer` marks concluding information. These elements help communicate structure to browsers, search tools, other developers, and assistive technologies.

Choose an element based on the meaning of its content rather than its default appearance. Semantic HTML is not a visual styling system: CSS controls appearance, while the element names express purpose. Use `section` for a thematically grouped part of a page, `article` for a self-contained composition, and `aside` for related content. A semantic outline makes a page easier to navigate and maintain.

Shortcut	Generated tag
<code>header</code>	<code><header></header></code>
<code>nav</code>	<code><nav></nav></code>
<code>main</code>	<code><main></main></code>
<code>section</code>	<code><section></section></code>
<code>article</code>	<code><article></article></code>
<code>aside</code>	<code><aside></aside></code>
<code>footer</code>	<code><footer></footer></code>
<code>figure</code>	<code><figure></figure></code>
<code>figcaption</code>	<code><figcaption></figcaption></code>
<code>details</code>	<code><details></details></code>
<code>summary</code>	<code><summary></summary></code>

7 Practical Patterns

Real interface components are usually made from several elements. Emmet lets you describe these structures compactly, then expand them into editable HTML. A navigation list, a card, a group of cards, or a login form can be drafted in one expression. These patterns are useful as starting points, particularly when the same basic structure will be repeated across a page.

The main risk of a compact expression is that it can hide assumptions. A card abbreviation that includes an image does not supply suitable alternative text. A login form does not automatically add labels or define how the form is handled. Treat each pattern as a skeleton: expand it, review the hierarchy, and complete the content, attributes, and behavior required by the actual interface.

Purpose	Emmet abbreviation	Structure created
Navigation bar	<code>nav>ul>li*4>a</code>	Navigation containing a list of four linked items
Card	<code>div.card>img+h2+p+button</code>	Card container with image, heading, paragraph, and button
Three cards	<code>div.card*3>h2+p+button</code>	Three card containers, each with heading, paragraph, and button
Login form	<code>form>label+input:email+label+input:password+button</code>	Form scaffold with two labels, two inputs, and a button

7.1 A quick editing checklist

Once an abbreviation expands, take a moment to review the result in context. First, check the nesting: is each child inside the intended parent, and are sibling elements in the right order? Next, replace placeholders, empty values, and generic labels with real content. Finally, check semantic and accessibility details such as image alternatives, explicit form labels, and meaningful link text.

For repeated structures, inspect one item before duplicating or styling the pattern further. A mistake in a repeated abbreviation can appear several times at once. If the expansion is not what you intended, edit the abbreviation and expand it again, or correct the HTML directly. Emmet is there to accelerate routine work; it should make careful authoring easier, not replace it.

8 Conclusion

Emmet turns a compact description into useful HTML scaffolding. The central ideas are straightforward: use element names for tags, dots and hashes for classes and IDs, operators for relationships, and repetition or text expressions for repeated content. Once those pieces are familiar, larger structures become easier to read and quicker to create.

The best workflow is simple: write a small abbreviation, expand it, and inspect the markup before moving on. Add the details that a shorthand cannot know, especially meaningful content, correct attributes, and accessible labels. Used this way, Emmet saves time without making the underlying HTML mysterious.

References

- [1] Microsoft. *Emmet in Visual Studio Code*. <https://code.visualstudio.com/docs/editor/emmet>
- [2] Emmet. *Emmet Documentation*. <https://docs.emmet.io/>
- [3] WHATWG. *HTML Living Standard*. <https://html.spec.whatwg.org/>